ConceptFormer v2: Token-Efficient Knowledge Injection into Frozen Language Models

Interim report and research program

Joel Barmettler[*]
University of Zurich
joel.barmettler@uzh.ch

[Advisor – to join]
University of Zurich

[Collaborator – to join]
University of Zurich

Abstract

We study *knowledge injection* into frozen large language models: encoding a knowledge-graph entity's 1-hop neighborhood into k continuous "concept tokens" that are spliced into the frozen model's input, replacing $\sim$100 tokens of retrieved text with $k \approx 8$. The encoder — a small Perceiver-style resampler over the frozen model's own label embeddings — is trained *label-free* by full-vocabulary KL self-distillation against the same frozen model reading the facts as text. Under a hardened evaluation protocol (full 14,266-question PopQA, leakage-free held-out splits, paired statistics), we find: (i) concept tokens beat question-aware text retrieval and LLM-written summaries by 2.6–3.2$\times$ at an 8-token budget, and an untrained-injection control is near floor, so the learned compression carries the effect; (ii) unseen-entity accuracy *scales with training data* (2.05$\times$ from 10k to 100k training entities, not yet saturated) but *anti-scales with frozen-model size at fixed data* (the injection margin roughly halves from 0.8B to 2B parameters); (iii) counterfactual edge-swap probes show the model genuinely *reads* the injected graph, increasingly so with k; and (iv) the same encoder can inject through a multimodal model's *vision-token* interface at parity with the text interface at k=8. The two scaling trends pull in opposite directions, making the data$\times$model-size interaction the central open question; we lay out the measurement program (up to 1M entities, 0.8B–397B backbones) designed to resolve it.

1 Introduction

Large language models access factual knowledge through two established channels. Knowledge stored *in the weights* is uneditable, expensive to refresh, and unreliably recalled for long-tail entities — recent evidence suggests frontier models *encode* 95–98% of long-tail facts but successfully *recall* only 25–33% of them [22]. Knowledge provided *as retrieved text* is faithful and editable but costs hundreds of context tokens per query, paid again on every request. A third channel is emerging in the literature: compressing knowledge into a few *continuous tokens* consumed directly by a frozen model [2, 3, 7, 13].

ConceptFormer operates in this third channel with a specific commitment: the unit of compression is a *knowledge-graph entity*. For each Wikidata entity we encode its 1-hop neighborhood — an

[*]Internal interim report prepared to onboard collaborators; not a submission. All numbers trace to Weights & Biases run ids and per-item evaluation dumps in the project repository (`joelbarmettlerUZH/ConceptFormer`, branch `conceptformer-v2`); see `docs/RESEARCH_FINDINGS.md` for the finding-by-finding evidence log.

unordered set of (property, neighbor) pairs — into k soft "concept tokens" that are spliced into the input embeddings of a *frozen* LLM. The encoder is amortized and inductive: an unseen entity requires one forward pass, no per-entity training. Training requires no labeled QA data; the student (frozen LLM reading k concept tokens) is distilled against the teacher (the *same* frozen LLM reading the facts as text) by full-vocabulary KL along the teacher's greedy answer path. A first version of this idea was validated on GPT-2 with rank-based metrics and published at The Web Conference 2026 companion track, where it received its hosting workshop's best paper award [1]; this project rebuilds it on modern backbones (Qwen3-0.6B; Qwen3.5-0.8B/2B) with a generative exact-match protocol, and asks the questions v1 could not: does the encoder *generalize* to unseen entities, does the model actually *read* the injected graph, and how does all of this *scale*?

This report consolidates roughly three months of experiments (about 60 training runs and a systematic evaluation overhaul) for new collaborators. It reads like a paper because most of it will become one, but we flag throughout which claims are established, which are single-pilot signals, and which experiments are planned but not yet run.

Summary of what we believe we know. (C1) At matched token budgets $\leq$32, concept tokens dominate every text baseline we could construct, including question-aware retrieval (§5.1). (C2) The effect is the *trained encoder*: untrained injection of the same embeddings is near floor. (C3) Unseen-entity generalization scales with training entities and had not saturated at 100k (§5.2). (C4) At fixed training data, the injection margin *shrinks* as the frozen backbone grows (§5.3). (C5) Counterfactual interventions show genuine, capacity-limited graph reading (§5.5). (C6) Injection preserves the frozen model's behavior off-topic. (C7) The recipe transfers across model families and attention architectures without retuning, and can enter through a vision-token interface at parity for small k (§5.4). (C8) At our evaluation scale, most single-run effect sizes below $\sim$4 points are measurement noise unless item-paired statistics are used; our corrected protocol (§4) is itself a reusable contribution.

2 Related work

Lineage (from v1). The problem framing descends from the observation that language models act as (unreliable) knowledge bases [33, 34], with the deficit concentrated on long-tail entities [35, 22]. The standard remedy is retrieval-augmented generation [32], which pays for faithfulness in context tokens. An older family of *knowledge-enhanced* PLMs injects KG information by modifying the model itself — KnowBERT [36], K-BERT [37], K-Adapter [38] — which our frozen-model constraint deliberately rules out. On the representation side, ConceptFormer v1 [1] injected *pretrained* KG node embeddings (TransE-family vectors at PyTorch-BigGraph scale [39, 40]) through a per-entity lookup table; v2 replaces this with features built from the frozen LLM's own label embeddings, which is what makes the encoder inductive (§3) — unseen entities need labels, not pretrained vectors. The mechanism of prepending *learned continuous tokens* to a frozen model descends from prefix-tuning and soft prompts [41, 25, 42], textual inversion's "an image is worth one word" [43], and, most directly, Frozen [44] and Flamingo [28], which feed continuous non-text tokens into frozen LMs — the precedent behind both our text and vision injection ports. Our substrate is Wikidata [45]; v1's evaluation data derives from T-REx [46], and graph verbalization baselines follow the graph-to-text encoding line [47].

Soft-token context compression. A line of work compresses *text* into few continuous tokens: gist tokens [4] and AutoCompressor [5] adapt the LM itself; ICAE [3] and 500xCompressor [6] train LoRA encoders against frozen decoders with reconstruction objectives; PISCO [7] uses sequence-level distillation but reports that frozen decoders underperform (their Appendix G) — a claim our frozen-decoder results contradict at entity granularity; CompLLM [8] distills per-segment "concept embeddings" via activation matching; ARC-Encoder [9] is the strongest recent frozen-decoder text compressor. Closest to us is xRAG [2], which shares our frozen decoder, input-embedding injection, and same-model KL self-distillation — but xRAG *translates* a single pre-existing retriever vector

rather than learning compression, is retriever-bound at inference, and evaluates neither unseen-entity generalization nor faithfulness. Cartridges [10] shares the distillation objective family but trains a KV cache *per corpus* by gradient descent: no amortization, no generalization to unseen items. None of these works compresses *structured* input, and none runs causal probes on the compressed representation; the only faithfulness-adjacent work is post-hoc overflow probing [11].

Knowledge-graph injection into LLMs. GNP [15], GraphToken [16], G-Retriever [17], and LLaGA [18] encode graphs into soft prompts for frozen LLMs, but all encode *per question* with task cross-entropy on labeled QA data, and none precomputes reusable per-entity tokens. KBLaM [13] precomputes one key–value pair *per triple* injected into every attention layer via modified attention — no neighborhood aggregation, synthetic-KB evaluation only. Knowledge Prompts [14] is the closest precompute precedent: per-entity soft prompts for 1.1M Wikidata entities — but as a lookup table of free parameters, so unseen entities are impossible by construction. KAPING [19] is the "facts as text" baseline family we compare against. Recent meta-analyses find that graph-token LLMs "do not fully understand graph tokens" [20] and that graph tokens can degenerate into attention sinks decoupled from graph semantics [21] — diagnoses that motivate exactly the counterfactual evaluation we build in (§5.5).

Positioning. No prior work combines: an *amortized, inductive* encoder producing k *query-independent* soft tokens per KG entity; *label-free same-model* full-vocab distillation; *unseen-entity* and *causal-faithfulness* evaluation; at 100k+ entity scale. Each neighbor has some of these properties; none has the conjunction. On the scaling side, knowledge-capacity laws exist for *parametric* storage [23, 24], and prompt-tuning quality is known to improve with model scale for *tasks* [25], but no study measures injection quality against frozen-model scale within one family, and no data-scaling law for a knowledge encoder exists. Those two empty cells are this project's target.

3 Method

Setup. Let M be a frozen chat LLM with input-embedding dimension d. For an entity e with 1-hop neighborhood $G_e = \{(p_i, n_i)\}_{i=1}^{N}$ (property and neighbor labels from Wikidata, rank-ordered by PageRank), the *featurizer* embeds each edge as $x_i = [\,\phi(p_i)\,;\,\phi(n_i)\,] \in \mathbb{R}^{2d}$, where ϕ mean-pools M's own input embeddings over the label's tokens. Features are thus inductive (any labeled entity works) and live in a space M already understands.

Encoder. A Perceiver/Flamingo-style latent-query resampler [27, 28]: k learned latent queries cross-attend over the edge set (masked, permutation-invariant), self-attend, and pass through a feed-forward block, for L layers (default d_{model}=1024, L=4, ∼70M parameters); an output projection maps to $\mathbb{R}^{k\times d}$. The output projection is *zero-initialized*, so at step 0 the student is bit-identical to the frozen model — capability preservation by construction (we found this variant, "gate-none", also more stable than a multiplicative tanh gate).

Objective: same-model KL self-distillation. The teacher is M reading the facts as text: [system][verbalized G_e][question], from which we store the greedy answer path $y_{1..m}$ offline. The student is M reading [system][$C_k(G_e)$][question], where C_k is the encoder output spliced into the input embeddings at a contiguous, unit-step RoPE span. The loss is the token-level full-vocabulary forward KL along the stored path,

$$\mathcal{L} \;=\; \frac{\tau^2}{m}\sum_{t=1}^{m} \mathrm{KL}\big(\, p_M(\cdot \mid \text{facts}, q, y_{<t}) \;\|\; p_M(\cdot \mid C_k(G_e), q, y_{<t})\,\big)\,, \tag{1}$$

with temperature τ=1. No QA labels are used anywhere in training; the corpus questions (generated by a separate LLM from the graphs; see the data pipeline below) exist only to give the teacher something to answer. Only the ∼20 path positions are materialized through the LM head, which makes training memory-light. The verbalized teacher context is answer-guaranteed (the questioned

edge is always present), which at our 2048-token budget changes almost nothing since a median neighborhood verbalizes to $\sim$100 tokens.

Injection ports. By default the concept block enters through the *text port*: spliced directly into the input-embedding stream at the position where the facts text sat for the teacher. For natively multimodal backbones (every Qwen3.5 size) we additionally implemented a *vision port*: the same k vectors enter as a pseudo-image — $[\texttt{vision\_start}][k \times \texttt{image\_token}][\texttt{vision\_end}]$ token ids, with the model's own M-RoPE grid positions for a $1 \times k$ patch layout, exactly as a real k-patch image would. The hypothesis: multimodal models are *pretrained* to consume continuous out-of-vocabulary token streams at these positions, which may make the vision interface a better "landing pad" for soft tokens than a text stream where every pretraining token was a row of the embedding matrix.

Data pipeline. From the Wikidata PageRank dump we sample entity pools (PopQA subjects excluded), snapshot complete 1-hop neighborhoods, generate QA with an independent LLM (Gemma via vLLM), *tier* each question by whether the graph demonstrably helps the backbone (base fails, facts-in-context succeeds), and store the teacher's greedy path. Corpora in play: 10k entities ($92{,}177$ distill rows under the Qwen3-0.6B teacher; $84{,}843$ under Qwen3.5-0.8B — a stronger base answers more parametrically, so its tiered corpus is smaller and harder), 100k entities ($925{,}178$ rows), a built 260k-entity snapshot, and a planned 1M. Note the consequence: *the corpus is teacher-relative*, so cross-backbone comparisons are of "each model with its own curriculum", not of identical training sets.

4 Evaluation protocol (and the pitfalls it fixes)

Every result is reported against two brackets from the same frozen model: **base** (no knowledge; floor) and **RAG** (facts as text, answer-guaranteed; ceiling, $\approx$0.94–0.96 on full PopQA). Metric: greedy exact-match with alias matching (we also report the official, more lenient PopQA metric; it tracks ours within $\sim$0.5 pt). Three accuracy axes: **held-out** (unseen questions about *trained* entities), **held-in** (trained questions; overfitting gauge), and **PopQA** (unseen *entities*; the honest headline, since the encoder must generalize inductively).

An external review mid-project identified four measurement defects that our earlier numbers carried; all are fixed, and all headline numbers in this report come from the corrected protocol. We document them because they are easy to reproduce in any similar project:

1. **Seed-coupled eval sampling.** Evaluation subsets were sampled with the training seed, so every run scored *different* $n{=}200$ questions; cross-seed spread conflated model variance with $\pm$3.5 pt binomial sampling noise. Fix: one frozen, documented eval seed for everything; smaller samples are prefixes of larger ones, so any two runs pair item-by-item.
2. **Tiny benchmark subsets.** $n{=}200$ of 14,267 PopQA questions. Fix: evaluate the *full* benchmark (binomial noise $\sim$0.4 pt); report Wilson intervals and exact McNemar tests on shared items. After this fix, apparent seed-to-seed standard deviations collapsed from 4–8 pt to 0.2–2.4 pt: most of what we had been calling run variance was measurement noise.
3. **Selection bias.** Best-checkpoint selection used the same $n{=}200$ sample later reported. Fix: definitive re-scoring on larger, disjointly-sampled sets.
4. **Paraphrase leakage.** Splitting by *question* let paraphrases of the same (entity, fact) straddle train/held-out — measured at **32.8%** of held-out rows. Fix: split by fact; filter legacy splits at eval time. Held-out numbers dropped 2–4 pt everywhere; orderings survived.

Additionally, the token-efficiency baseline was strengthened from a single query-independent truncation to three retrieval modes (§5.1) to pre-empt the strawman objection, and an untrained-injection control isolates the encoder's contribution.

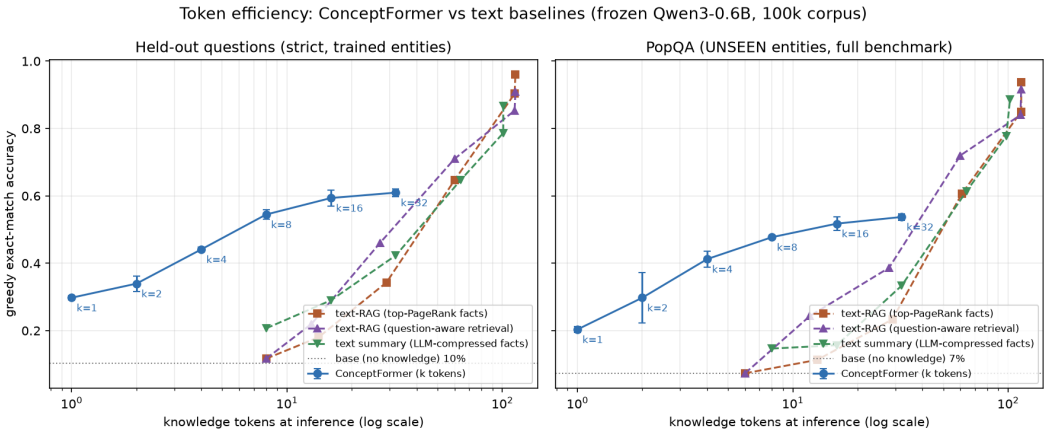

Figure 1: Accuracy vs. knowledge tokens spent at inference (log scale), frozen Qwen3-0.6B, 100k-entity corpus. Concept tokens (blue, mean±std over 3 seeds) dominate all three text baselines below ∼32 tokens on both axes; text catches up only at 64–128 tokens. Left: strict held-out; right: full PopQA (unseen entities).

Table 1: Corrected k-family on the 100k-entity corpus (frozen Qwen3-0.6B; 3 seeds; mean±std). Strict held-out n=2000; full PopQA n=14,266. Brackets: base 0.103, RAG 0.960. The curve is monotone through k=32; all adjacent contrasts are significant under item-paired McNemar.

k	1	2	4	8	16	32
held-out	.298±.006	.340±.023	.441±.007	.545±.014	.594±.024	.610±.011
PopQA	.203±.008	.298±.075	.412±.023	.477±.002	.518±.020	.537±.010

5 Results

5.1 Token efficiency: concept tokens vs. text at matched budgets

Figure 1 shows the central comparison. At an 8-token budget on unseen entities, concept tokens reach 0.477 where the *best* text baseline (an LLM-written budgeted summary) reaches 0.147, and question-aware retrieval — which may select facts *after seeing the question*, an advantage our query-independent tokens do not have — reaches 0.073; at 16 tokens the gaps remain 2× or larger. Text needs 64–128 tokens (most of a verbalized neighborhood; median 100 tokens) to catch up, i.e., matched accuracy costs text ∼5–6× more tokens. Two controls make this meaningful. First, the **untrained-injection control**: filling the same k slots with mean-pooled embeddings of the top-k edges scores 0.130 on PopQA (base 0.103), identically for k=8 and k=16 — the learned compression carries ∼93% of the injected-knowledge effect. Second, the corrected k-curve (Table 1) rises monotonically through k=32 with every adjacent step significant under paired McNemar (e.g. $k16{\to}k32$: $p{\approx}10^{-38}$) — our earlier belief in a "knee at k=16" was an n=200 artifact. Returns per token diminish: k=8 buys ∼89% of k=32's PopQA at a quarter of the tokens.

5.2 Data scaling: more entities, better generalization

Scaling the training corpus from 10k to 100k entities (identical eval sets, k=8, Qwen3-0.6B) lifts full-PopQA accuracy from 0.232 ± 0.011 to 0.477 ± 0.002 — a 2.05× gain in unseen-entity generalization, still climbing at the end of training. Under the strict protocol, held-out *also* rises ($0.453 \to 0.545$); our earlier belief that held-out was flat across scales was a leakage artifact (the 25-epoch 10k models overfit paraphrases more). Meanwhile held-in *fell* (∼0.82 →∼0.64): the model memorizes training entities less and learns the edge→concept mapping more — the signature of graph-learning rather than corpus memorization. In margin terms, the concept-over-base margin on unseen entities grows from +12.9 to +37.4 points with 10× data.

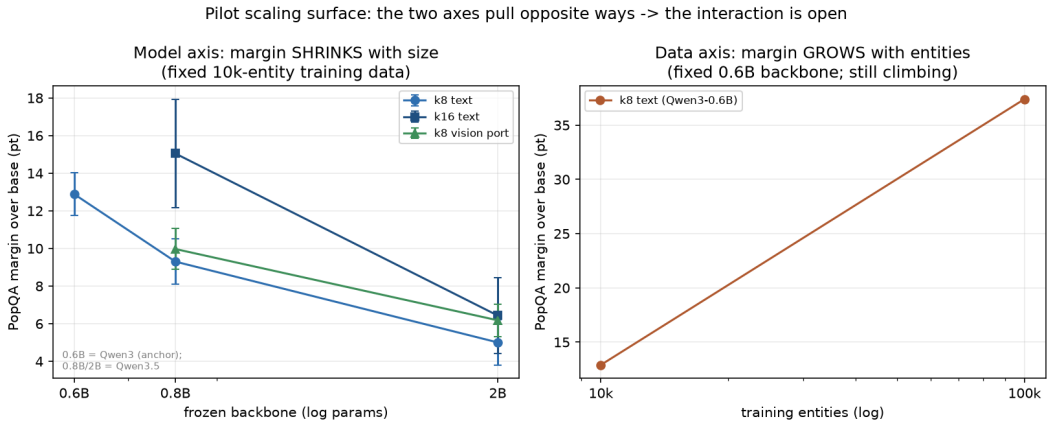

Figure 2: The pilot scaling surface. Left: at fixed training data (10k entities), the PopQA injection margin over each backbone's own base *shrinks* as the frozen model grows (0.6B is the Qwen3 anchor; 0.8B/2B are Qwen3.5, 2 seeds each). Right: at a fixed backbone (0.6B), the margin *grows* with training entities. The two axes pull in opposite directions; their interaction is unmeasured and is the central open question.

5.3 Model scaling at fixed data: the margin anti-scales

To probe the model axis we migrated to the Qwen3.5 family (natively multimodal at every size) and ran a two-point transect at fixed 10k-entity data: 0.8B and 2B backbones, $k \in \{8, 16\}$, two seeds each, full-benchmark evaluation (Figure 2, left). Each backbone gets its own teacher-relative corpus and brackets. The result: the PopQA margin over base falls from $+12.9$ pt (0.6B anchor) to $+9.4$ (0.8B) to $+5.1$ (2B) at k=8, and roughly halves from 0.8B to 2B at k=16 ($+15.1 \rightarrow +6.5$) — while strict held-out stays healthy at 2B (0.465–0.503), so the encoder still works; what shrinks is its *marginal value on unseen entities*. Two interpretations are live and the data cannot yet separate them: a *ceiling effect* (larger bases know more of PopQA: base rises $0.103 \rightarrow 0.124 \rightarrow 0.150$) versus a *steerability effect* (a larger frozen model needs more training signal per unit of behavioral change from k soft tokens). The 100k-corpus corners of the planned surface (§6) are designed to disentangle them. Migration itself was uneventful: the locked recipe (optimizer, learning rate, batch, capacity) transferred across families and across an attention-architecture change (Qwen3.5 is a hybrid linear-attention model) with zero retuning — and, notably, the two-point transect already shows the model axis is *not* a free win, which is precisely why it needs to be measured rather than assumed.

5.4 Injection port: text stream vs. vision stream

On the same pilots we compared the text port against the vision port (item-paired, same frozen eval sets). At k=8 the ports are at **parity** on PopQA at both sizes (deltas -0.9/$+2.4$ pt at 0.8B, -0.3/$+2.7$ at 2B; signs vary by seed) and near-parity on held-out. At k=16 (tested at 0.8B) the vision port *trails* (-2.8 and -7.0 pt held-out, $p \leq 4 \times 10^{-3}$), with one seed clearly unstable. So the "vision tokens are a better landing pad" hypothesis is *not* supported at these scales — but the ports demonstrably learn different solutions (up to 1,800 discordant items per cell), the vision port received no port-specific tuning, and vision pretraining strength grows sharply with model size in this family. The port$\times$scale interaction at 27B+ remains open; the pilot's value is converting it from speculation into a measured, non-trivial question, with all mechanics (M-RoPE grid positions via the model's own indexer, KV-cached manual decoding) already built and tested.

5.5 Does it read the graph? Counterfactual faithfulness

A compression method can succeed on QA while merely memorizing text patterns. Following the deficiency diagnosed across graph-token systems [20, 21], we built causal probes in: (a) *edge ab-*

Table 2: Causal graph probes (Qwen3-0.6B, 100k corpus, 400 single-fact probes per k, one seed). "Swap-follow": after rewiring the answer edge to a type-plausible false neighbor, the model outputs the *false* answer — the graph-reading signature; it scales with k. Ablating the answer edge collapses accuracy while ablating an unrelated edge does not: edges are encoded separably. The frozen base without concepts knows only 10% of these probes.

k	1	4	8	16	32
concept correct	41%	56%	63%	65%	70%
ablate answer edge (want low)	55%	38%	34%	36%	28%
ablate other edge (want high)	90%	96%	92%	95%	94%
swap-follow → false answer	0.8%	16.5%	25.8%	29.8%	34.2%

lation — removing the questioned edge should collapse that answer while removing an unrelated edge should not; (b) *counterfactual swap* — rewiring the answer edge to a plausible false neighbor should flip the model to the *false* answer if and only if it reads the graph rather than its parametric memory (the frozen base knows only 10% of these probes, so memory is controlled). Table 2: edges are separably encoded at every k, and swap-following rises monotonically from 0.8% (k=1) to 34.2% (k=32) — faithfulness is *capacity-limited, not absent*, and becomes another axis that scale buys. On the 10k-corpus Qwen3.5-0.8B pilots, swap-follow is much lower (5–13%) — confounded with corpus size and training regime, but consistent with faithfulness being the slowest-developing property. Off-topic behavior is preserved: on control tasks that mention the entity without asking about its facts, injecting concepts moves the next-token distribution by a median KL of only 0.06–0.08 nats at 0.6B/100k, flat in k — by construction (zero-initialized output projection) the student starts bit-identical to the frozen model. The 10k-corpus pilots show larger control-task perturbation ($\sim$0.27), another regime effect to track at scale.

5.6 Optimization and stability notes

For onboarding completeness: training stability was an early obstacle ($\sim$8 pt outcome spread between nominally identical runs). The stabilized recipe — drop the multiplicative tanh gate in favor of a zero-initialized output projection, effective batch 32 via gradient accumulation, learning rate 10^{-4} held constant — brought the spread to the $\sim$1 pt range *as measured then*; the corrected protocol later showed much of the residual spread was evaluation noise. Two honest caveats survive: our batch-size comparisons at fixed step counts confound batch size with data passes (a larger effective batch sees more data before the fixed stop), so we treat "effective batch 32" as a validated default rather than a mechanism claim; and learning-rate rankings from short-horizon sweeps systematically favor small rates — we only trust horizon-matched comparisons.

6 Open questions and planned experiments

Q1 — The data×model interaction (central). Does a larger frozen model simply need more entities before injection pays, or is it intrinsically harder to steer? Planned: the full surface — entities $\{10\text{k}, 100\text{k}, 300\text{k}, 1\text{M}\}$ × Qwen3.5 $\{0.8, 2, 4, 9, 27\}$B dense, k-family, 3 seeds — with the 100k×{9B, 27B} corners first, since they directly separate the ceiling from the steerability hypothesis. The 260k-entity corpus is built; 1M is pipeline-ready. *Status: not yet run; requires compute beyond our two 24 GB GPUs for every backbone above $\sim$4B.* **Q2 — MoE: total or active parameters?** Qwen3.5's MoE members (35B-A3B, 122B-A10B, 397B-A17B) let us ask which parameter count governs injection. *Not yet run.* **Q3 — Port×scale.** Does the vision port's parity-at-k=8 become an advantage at sizes with strong vision pretraining? *Mechanics built; larger sizes not yet run.* **Q4 — Faithfulness at scale.** Does swap-following keep rising with data and k, and can an auxiliary per-edge objective accelerate it? *Design sketched, not yet run.* **Q5 — Missing baselines.** An xRAG-style retriever-vector bridge trained under our exact protocol, per-entity prefix-tuning [41] (the transductive upper baseline, and the closest modern analogue to v1's lookup table), and on-policy distillation (GKD [30]) as an objective ablation. *Not yet run.* **Q6 — Beyond 1-hop.** 2-hop

neighborhoods raise the facts-per-token ratio $\sim$10–100$\times$ at fixed k; we treat this as a rate–distortion ablation inside Q1, not a standalone contribution (the per-question subgraph-encoding literature is crowded [15, 17]). **Q7 — Second benchmark.** EntityQuestions integration for a second unseen-entity axis. *Loader stubbed.*

A compute-grant application (Swiss AI Initiative, CSCS Alps) is drafted to fund Q1–Q3; the pilot surface (Figure 2) is its preliminary-results centerpiece.

7 Reproducibility and infrastructure

Everything is public in the project repository: the data pipeline (snapshot, generation, tiering, teacher-path extraction; all resumable and cached), training (single 24 GB GPU per run), and the evaluation harness. Every metric in this report traces to a W&B run id (entity `university-of-zurich`, project `conceptformer-v2`) or a re-runnable CLI command, and every evaluation writes *per-item* outputs so any two systems can be compared with paired statistics after the fact. The finding-by-finding evidence log (`docs/RESEARCH_FINDINGS.md`, F1–F16 plus methods notes M1–M7) records not only results but also the claims we *retracted* and why — start there, then `docs/RELATED_WORK.md` (annotated survey behind §2) and `docs/GRANT_PROPOSAL_DRAFT.md`. The three-command gate (`ruff`, `ty`, `pytest`; 244 tests) is green at every commit.

Acknowledgments and Disclosure of Funding

This interim report was prepared with substantial tooling assistance from Claude (Anthropic) for literature search, evaluation-harness engineering, and drafting; all experimental design decisions, runs, and claims were reviewed by the author. No external funding to declare yet — that is what §6 is for.

References

[1] J. Barmettler, A. Bernstein, and L. Rossetto. ConceptFormer: Towards graph-native grounding of large language models via latent concept injection. In *Companion Proceedings of the ACM Web Conference 2026 (WWW Companion '26)*, pp. 587–596, 2026. Best paper award, hosting workshop. `https://doi.org/10.1145/3774905.3794653`. arXiv:2504.07624.

[2] X. Cheng, X. Wang, X. Zhang, T. Ge, S. Chen, F. Wei, H. Zhang, and D. Zhao. xRAG: Extreme context compression for retrieval-augmented generation with one token. NeurIPS 2024; arXiv:2405.13792.

[3] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, and F. Wei. In-context autoencoder for context compression in a large language model. ICLR 2024; arXiv:2307.06945.

[4] J. Mu, X. L. Li, and N. Goodman. Learning to compress prompts with gist tokens. NeurIPS 2023; arXiv:2304.08467.

[5] A. Chevalier, A. Wettig, A. Ajith, and D. Chen. Adapting language models to compress contexts. EMNLP 2023; arXiv:2305.14788.

[6] Z. Li, Y. Su, and N. Collier. 500xCompressor: Generalized prompt compression for large language models. arXiv:2408.03094, 2024.

[7] M. Louis, G. Sidiropoulos, E. Kanoulas, and V. Nikoulina. PISCO: Pretty simple compression for retrieval-augmented generation. arXiv:2501.16075, 2025.

[8] G. Berton et al. CompLLM: Compression for long context Q&A. arXiv:2509.19228, 2025.

[9] H. Dezéque et al. (Kyutai). ARC-Encoder: Learning compressed text representations for large language models. arXiv:2510.20535, 2025.

[10] S. Eyuboglu et al. Cartridges: Lightweight and general-purpose long context representations via self-study. arXiv:2506.06266, 2025.

[11] Anonymous. Detecting overflow in compressed token representations for retrieval-augmented generation. arXiv:2602.12235, 2026.

[12] Z. Pan et al. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. ACL Findings 2024; arXiv:2403.12968.

[13] X. Wang, T. Isazawa, L. Mikaelyan, and J. Hensman. KBLaM: Knowledge base augmented language model. ICLR 2025; arXiv:2410.10450.

[14] C. N. dos Santos, Z. Dong, D. Cer, J. Nham, S. Shakeri, J. Ni, and Y.-H. Sung. Knowledge prompts: Injecting world knowledge into language models through soft prompts. arXiv:2210.04726, 2022.

[15] Y. Tian, H. Song, Z. Wang, H. Wang, Z. Hu, F. Wang, N. V. Chawla, and P. Xu. Graph neural prompting with large language models. AAAI 2024; arXiv:2309.15427.

[16] B. Perozzi, B. Fatemi, D. Zelle, A. Tsitsulin, M. Kazemi, R. Al-Rfou, and J. Halcrow. Let your graph do the talking: Encoding structured data for LLMs. arXiv:2402.05862, 2024.

[17] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi. G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. NeurIPS 2024; arXiv:2402.07630.

[18] R. Chen, T. Zhao, A. Jaiswal, N. Shah, and Z. Wang. LLaGA: Large language and graph assistant. ICML 2024; arXiv:2402.08170.

[19] J. Baek, A. F. Aji, and A. Saffari. Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. arXiv:2306.04136, 2023.

[20] Anonymous. Revisiting graph-tokenizing LLMs: A systematic evaluation (GTEval). arXiv:2605.03514, 2026.

[21] Anonymous. When graph tokens sink: A mechanistic analysis of graph-token utilization in LLMs. arXiv:2606.03712, 2026.

[22] Anonymous. Empty shelves or lost keys? Disentangling encoding from recall of long-tail facts in language models. arXiv:2602.14080, 2026.

[23] Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.3, knowledge capacity scaling laws. arXiv:2404.05405, 2024.

[24] V.-P. Berges, B. Rozière et al. Memory layers at scale. arXiv:2412.09764, 2024.

[25] B. Lester, R. Al-Rfou, and N. Constant. The power of scale for parameter-efficient prompt tuning. EMNLP 2021; arXiv:2104.08691.

[26] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. ACL 2023; arXiv:2212.10511.

[27] A. Jaegle, F. Gimeno, A. Brock, A. Zisserman, O. Vinyals, and J. Carreira. Perceiver: General perception with iterative attention. ICML 2021; arXiv:2103.03206.

[28] J.-B. Alayrac et al. Flamingo: A visual language model for few-shot learning. NeurIPS 2022; arXiv:2204.14198.

[29] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[30] R. Agarwal, N. Vieillard, Y. Zhou, P. Stanczyk, S. Ramos, M. Geist, and O. Bachem. On-policy distillation of language models: Learning from self-generated mistakes. ICLR 2024; arXiv:2306.13649.

[31] Qwen Team. Qwen3 and Qwen3.5 model families. Technical reports, 2025–2026; `https://huggingface.co/Qwen`.

[32] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 2020.

[33] F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language models as knowledge bases? EMNLP 2019.

[34] A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? EMNLP 2020.

[35] K. Sun, Y. E. Xu, H. Zha, Y. Liu, and X. L. Dong. Head-to-tail: How knowledgeable are large language models? A.k.a. will LLMs replace knowledge graphs? arXiv:2308.10168, 2023.

[36] M. E. Peters, M. Neumann, R. L. Logan IV, R. Schwartz, V. Joshi, S. Singh, and N. A. Smith. Knowledge enhanced contextual word representations. EMNLP 2019.

[37] W. Liu, P. Zhou, Z. Zhao, Z. Wang, Q. Ju, H. Deng, and P. Wang. K-BERT: Enabling language representation with knowledge graph. AAAI 2020.

[38] R. Wang, D. Tang, N. Duan, Z. Wei, X. Huang, J. Ji, G. Cao, D. Jiang, and M. Zhou. K-Adapter: Infusing knowledge into pre-trained models with adapters. Findings of ACL 2021.

[39] A. Bordes, N. Usunier, A. García-Durán, J. Weston, and O. Yakhnenko. Translating embeddings for modeling multi-relational data. NeurIPS 2013.

[40] A. Lerer, L. Wu, J. Shen, T. Lacroix, L. Wehrstedt, A. Bose, and A. Peysakhovich. PyTorch-BigGraph: A large scale graph embedding system. MLSys 2019.

[41] X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. ACL 2021.

[42] G. Qin and J. Eisner. Learning how to ask: Querying LMs with mixtures of soft prompts. NAACL 2021.

[43] R. Gal, Y. Alaluf, Y. Atzmon, O. Patashnik, A. H. Bermano, G. Chechik, and D. Cohen-Or. An image is worth one word: Personalizing text-to-image generation using textual inversion. ICLR 2023.

[44] M. Tsimpoukelli, J. Menick, S. Cabi, S. M. A. Eslami, O. Vinyals, and F. Hill. Multimodal few-shot learning with frozen language models. NeurIPS 2021.

[45] D. Vrandečić and M. Krötzsch. Wikidata: A free collaborative knowledgebase. Communications of the ACM, 57(10):78–85, 2014.

[46] H. ElSahar, P. Vougiouklis, A. Remaci, C. Gravier, J. Hare, F. Laforest, and E. Simperl. T-REx: A large scale alignment of natural language with knowledge base triples. LREC 2018.

[47] B. Fatemi, J. Halcrow, and B. Perozzi. Talk like a graph: Encoding graphs for large language models. ICLR 2024; arXiv:2310.04560.